

JS Fundamentals

Variables, Types, Operators & Control Flow

■ JS BASICS

■ LEARNING OUTCOME

By the end of this module you will write JS confidently: declare variables with let/const, work with all primitive data types, use operators, write conditionals and loops, and understand type coercion.

■ Skills You Will Gain

- Understand let, const, var differences
- Work with string, number, boolean, null, undefined
- Use comparison: == vs === (always ===)
- Write if/else, switch, ternary operator
- Use all loop types: for, while, for...of, for...in
- Understand truthy/falsy values

■ JavaScript is the only language that runs natively in browsers. It powers 98% of all websites and is the foundation for React, Node.js, and every modern web framework.

SECTION 1

Variables: let, const, var

Golden Rule: const by default, let when reassigning, never var

const: Cannot be reassigned after declaration. Block-scoped. Use for 99% of variables. let: Can be reassigned. Block-scoped. Use when value changes (counters, toggles). var: Function-scoped, hoisted, can be redeclared. AVOID — causes subtle bugs. Block scope means the variable only exists inside the {} block where declared.

```
const PI = 3.14159; const API_URL = 'https://api.example.com'; const user = { name: 'Alice', age: 28 }; // const object -- properties CAN change! let score = 0; let isLoggedIn = false; score = 10; // reassignment OK with let isLoggedIn = true; // BLOCK SCOPE if (true) { const blockConst = 'inside block only'; let blockLet = 'also inside only'; var leaked = 'I LEAK outside!'; } // blockConst -- ReferenceError (block-scoped) // blockLet -- ReferenceError (block-scoped) // leaked -- accessible! (var ignores block scope) // Template literals (backtick strings) const name = 'Alice'; const score2 = 95; console.log(`Hello, ${name}! Score: ${score2}/100`); console.log(`2 + 2 = ${2 + 2}`); // expressions inside ${}
```

SECTION 2

Data Types

Type	Description	Examples
string	Text in quotes	'hello', "world", `template \${var}`
number	All numbers (int+float)	42, 3.14, -7, Infinity, NaN
boolean	True or false	true, false
null	Intentional empty value	let user = null; (you set it)
undefined	Variable not assigned	let x; // x is undefined
symbol	Unique identifier	Symbol('id') -- used for object keys
bigint	Integers beyond 2^53	9007199254740993n
object	Key-value collection	{name:'Alice',age:28}
array	Ordered list (special object)	[1,'two',true,{x:3}]

```
// typeof operator typeof 42 // 'number' typeof 'hello' // 'string' typeof true // 'boolean' typeof undefined // 'undefined' typeof null // 'object' -- famous JS bug! null is NOT an object typeof [] // 'object' -- use Array.isArray() instead typeof function(){} // 'function' // FALSY values (6 only): false, 0, '', null, undefined, NaN // TRUTHY: everything else -- including [], {}, '0', 'false' // Type conversion Number('42') // 42 Number('') // 0 Number('abc') // NaN String(42) // '42' Boolean(0) // false Boolean([]) // true -- empty array is TRUTHY! // Nullish coalescing ?? -- only for null/undefined const name2 = user?.name ?? 'Guest'; // 'Guest' only if null/undefined const count = data.count ?? 0; // NOT triggered by 0 or '' (unlike ||) // Optional chaining ?. -- safe nested access const city = user?.address?.city; // undefined instead of TypeError
```

SECTION 3

Operators and Control Flow

```
// === strict equality ALWAYS (never ==) 5 === 5 // true 5 === '5' // false -- different types 5 == '5' // true -- coerces type, AVOID // Logical operators true && false // false (AND) true || false // true (OR) !true // false (NOT) // if / else if / else const s = 85; if (s >= 90) { console.log('A'); } else if (s >= 80) { console.log('B'); } else { console.log('C'); } // Ternary const grade = s >= 90 ? 'A' : s >= 80 ? 'B' : 'C'; const status = isLoggedIn ? 'Welcome!' : 'Please log in'; // for...of -- iterate array VALUES (use this for arrays) const fruits = ['apple', 'banana', 'cherry']; for (const fruit of fruits) { console.log(fruit); } // for...in -- iterate object KEYS (use this for objects) const p = { name: 'Alice', age: 28 }; for (const key in p) { console.log(`${key}: ${p[key]}`); } // break and continue for (let i=0; i<10; i++) { if (i===3) continue; // skip 3 if (i===7) break; // stop at 7 console.log(i); // 0 1 2 4 5 6 }
```

■ PRACTICE Q & A

Q1. What is the difference between let, const, and var?

A: const: cannot be reassigned, block-scoped -- use by default. let: can be reassigned, block-scoped. var: function-scoped, hoisted, avoid -- causes unexpected bugs.

Q2. What is the difference between == and === in JavaScript?

A: === (strict): compares value AND type -- 5 === '5' is false. == (loose): converts types before comparing -- 5 == '5' is true. Always use ===.

Q3. What are the 6 falsy values in JavaScript?

A: false, 0, " (empty string), null, undefined, NaN. Everything else is truthy -- including [], {}, and '0'.

Q4. What does optional chaining ?. do?

A: Safely accesses nested properties without throwing TypeError if an intermediate value is null or undefined. user?.address?.city returns undefined instead of crashing.

Q5. What is the difference between ?? and ||?

A: || returns the right side for ANY falsy value (0, "", false, null, undefined). ?? returns right side ONLY for null or undefined. Use ?? when 0 or empty string are valid values.

■ JS Fundamentals Cheat Sheet	
const / let	const default; let when reassigning; no var
typeof value	Check type: 'string', 'number', 'boolean'
=== vs ==	Always strict ===, never loose ==
?? nullish	Default only for null/undefined
?. optional chain	Safe nested access
Falsy: false, 0, '', null, undefined, NaN	6 falsy values
for...of arr	Iterate array values
for...in obj	Iterate object keys
template `\${var}`	String interpolation
Number('42')	String to number
Boolean(val)	Truthy/falsy check
2**10 = 1024	Exponentiation operator